

A Retrieval-Augmented Multi-Agent System for Personalized Academic Scheduling

Martin Koome¹ and Mohamed Awud¹

¹Carnegie Mellon University, {mkoome, mawud}@andrew.cmu.edu

Group 14 | Agentic AI: Foundations and Applications | February 19, 2026

Abstract

Effective academic time management extends beyond basic calendar availability and requires awareness of individual energy patterns, workload intensity, deadlines, and evolving user preferences. This paper presents a Retrieval-Augmented Generation (RAG) based multi-agent scheduling system that learns personalized study and planning strategies across sessions. The architecture is organized around a Manager agent that functions as a central orchestrator, coordinating three specialized agents Planner, Executor, and Reviewer through a shared persistent memory implemented in Supabase (PostgreSQL with pgvector). Semantic retrieval over a domain-specific time management knowledge base, together with adaptive caching and groundedness verification, enables the system to generate context-aware scheduling recommendations grounded in chronotype, course difficulty, workload constraints, and historical behavior. An independent evaluation framework of 40 structured scenarios demonstrates reliable integration of retrieval and calendar tools, accurate conflict detection, strong personalization fidelity, and effective mitigation of hallucinated outputs. The results indicate that persistent memory, structured retrieval, and modular agent coordination provide a practical foundation for trustworthy and context-aware academic scheduling assistants.

Architecture Evolution

The architecture evolved from an early multi-agent scheduling prototype with local vector retrieval and limited session memory to a centrally orchestrated system with persistent shared memory. In the initial implementation, multiple agents performed planning and calendar interaction over a ChromaDB retrieval layer, but coordination was implicit and state remained largely session-bound. Planning, option presentation, and execution were only loosely separated, and agents relied primarily on immediate conversational context rather than durable cross-session knowledge.

The current system introduces a Manager agent that functions as a centralized orchestrator coordinating specialized Planner, Executor, and Reviewer agents through a shared Supabase memory layer. The migration from ChromaDB to Supabase (PostgreSQL with pgvector) enables unified relational and vector storage for conversations, preferences, prior plans, and performance metrics. An adaptive control layer further wraps agent tools with caching, retry logic, groundedness verification, and confidence-based clarification. Together, these changes establish explicit orchestration boundaries, persistent personalization, and more reliable scheduling execution.

Memory Design Rationale

The system employs a persistent memory architecture implemented in Supabase (PostgreSQL with pgvector) to support cross-session personalization, retrieval-augmented reasoning, and multi-agent coordination. This enables agents to store and query structured entities such as sessions, preferences, and plans while also performing vector similarity search over user profile embeddings.

The memory layer is organized into nine tables capturing complementary aspects of scheduling intelligence, including conversation history, user preferences, prior plans, agent context, and performance metrics. Write policies persist all interactions, tool usage logs, and confirmed scheduling plans, ensuring that learned behaviors accumulate over time. Read policies prioritize recent conversational context and high-confidence preferences to guide planning decisions. Pruning policies maintain efficiency by summarizing older sessions and retaining only reusable or highly rated plans.

This design enables indirect agent coordination through shared state while preserving long-term personalization and bounded retrieval context.

Evaluation Metrics and Results

System performance was evaluated using a comprehensive benchmark of 40 structured scenarios across four categories: time logic and conflict detection, RAG-based retrieval, calendar tool integration, and multi-tool coordination. Each scenario measured groundedness, conflict detection accuracy, and overall task success. Scenarios were executed end-to-end through the Manager agent orchestrator, with agents retrieving context from the Supabase memory and executing actions under human-in-the-loop constraints.

Category	Number of Scenarios	Pass Rate
Time Logic & Conflict Detection	10	90%
RAG System Retrieval	10	80%
Calendar Tool Integration	10	100%
Multi-Tool Integration	10	80%
Overall	40	87.5%

Table 1: Evaluation results across all scenario categories.

Adaptive control mechanisms improved efficiency, yielding an average cache hit rate of 45% and saving approximately 900 tokens per session. Groundedness scores averaged 0.875, demonstrating that the system consistently produced context-aware and memory-supported scheduling decisions.

Failure Case Deep Dive

While testing the scheduling system, I observed a critical failure when I selected schedule options using numeric inputs, such as “4” for the fourth available time. Instead of proceeding to plan creation and execution, the system repeatedly called the manager agent, which rechecked the calendar and presented the same options. This behavior caused an infinite loop, preventing task completion, generating unnecessary API calls, and resulting in user frustration.

Further investigation during testing revealed that the system only recognized a limited set of confirmation words, such as “yes” or “correct”. Numeric replies were treated as unclear and routed back to the manager agent. Additionally, the manager agent lacked explicit instructions to stop after presenting options, while the planner agent could not interpret numeric selections from previous responses. Other contributing factors included a limited set of confirmation keywords and no tracking of options that had already been presented.

To address the issue, I implemented numeric input detection for values one through ten and added context checks to determine whether options had already been displayed. The manager agent was updated to stop after presenting options, delegating selection handling to the planner agent. The planner was enhanced to read the chosen option from the previous response and extract the corresponding time and date, eliminating repeated prompts. Confirmation keywords were expanded and conversation memory tracking was improved to maintain state across interactions.

After applying these changes and testing extensively, scheduling flows completed reliably in three agent steps: manager, planner, and executor. Infinite loops were eliminated, calendar API calls decreased by more than sixty percent, and numeric selections were correctly handled in all test cases. Task completion times became consistent, and the user experience improved significantly. Overall, the system now handles numeric and natural language inputs effectively and coordinates agent interactions efficiently, providing a smoother and more reliable scheduling experience.

Scalability Reflection

If deployed to approximately 1,000 concurrent users, the first bottleneck would likely arise in the Supabase memory layer. Frequent vector retrieval and relational writes from multiple agents would increase database contention and latency. Although indexing and caching mitigate load, sustained concurrency would require horizontal database scaling and distributed caching to maintain response times.

In high-stakes domains such as healthcare or financial scheduling, the primary limitation would be decision reliability. The Planner agent remains probabilistic and may generate suboptimal recommendations under complex or safety-critical constraints. Additional deterministic rules, domain validation layers, and formal verification would be required before deployment in such environments.

Within regulated industries, data governance would present the earliest failure point. Persistent storage of user preferences, behavior, and calendar data introduces privacy, retention, and audit requirements. The current system lacks role-based access control, encryption policies, and compliance audit trails required by regulatory frameworks. Addressing these constraints would require secure storage policies, access controls, and explainable decision logging.

Overall, while the architecture demonstrates robust performance for personalized academic scheduling, scaling to large populations or regulated contexts would require enhanced database scalability, deterministic safety constraints, and compliance-aware data management.

Contribution Statement

This project was completed collaboratively with clearly defined responsibilities. Martin Koome led the design and implementation of the multi-agent architecture, developed the core agent logic and tool integration, implemented the Google Calendar interface, and conducted extensive system testing, evaluation, and failure analysis, including detailed investigation and resolution of infinite loop issues during numeric option selection.

Mohamed Awud focused on the design and implementation of the Retrieval-Augmented Generation system, including advanced semantic chunking strategies, metadata-driven retrieval, and optimization of personalized data access. We jointly contributed to system design decisions, evaluation methodology, performance analysis, and technical documentation.

Future Work

Building on the current multi-agent scheduling system, future work will focus on further improving performance, personalization, and deployment. While the system now reliably handles numeric selections, coordinates multiple agents, and completes tasks efficiently, latency remains an area for improvement. Optimizing agent response times and streamlining tool integrations will provide a faster and smoother user experience.

To enhance usability, a small user interface has been developed using Streamlit, providing an accessible and intuitive way for users to interact with the system. Future work will continue to refine agent specialization for long-term planning and adaptive scheduling. A Recovery Agent could propose adaptive plans to compensate for lost or unproductive time, improving overall efficiency.

From a deployment perspective, fully containerizing the system will simplify installation, support open-source distribution, and allow easier integration with other platforms. Future iterations will also investigate the use of open-source language models to enhance privacy, transparency, and flexibility. These improvements aim to create a faster, more adaptable, and user-centered academic planning platform while enabling ongoing development and community contribution.

The current project codebase, including the implementation and evaluation framework, is available at My Life in Blocks Repository, providing a foundation for future enhancements and experimentation.